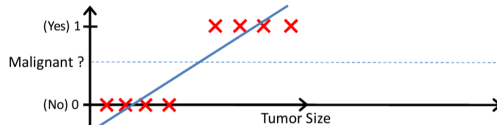


- 1 Introduction
- 2 Logistic Regression
- 3 Summary
- 4 Extra reading
- 5 References

- 1 Introduction
- 2 Logistic Regression
- 3 Summary
- 4 Extra reading
- 5 References

Classification problem (cont.)

- Can we solve the problem using linear regression?



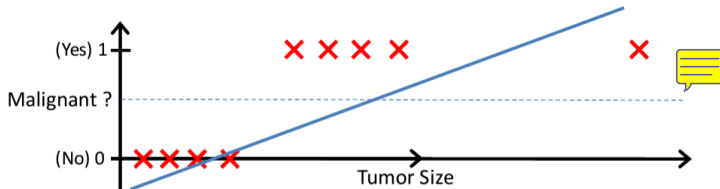
- We could fit a straight line and define a threshold at 0.5:

If $h_{\theta}(x) \geq 0.5$, predict $y = 1$

If $h_{\theta}(x) < 0.5$, predict $y = 0$

Classification problem (cont.)

- What about now? (By adding a new data point)



- Classification: $y = 0$ or $y = 1$
 - $h_{\theta}(x)$ can be > 1 or < 0
 - Another drawback of using linear regression for this problem
- What we need:

Logistic regression: $0 \leq h_{\theta}(x) \leq 1$

- We also show this function with other notations: $f(x; w) = \sigma(w^T x)$

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

③ Summary

4 Extra reading

5 References

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

③ Summary

4 Extra reading

5 References

Introduction (cont.)

- $\sigma(w^T x)$: probability that $y = 1$ given x (parameterized by $\mathbf{w}$)

$$P(y = 1|x, \mathbf{w}) = \sigma(\mathbf{w}^T x)$$

$$P(y = 0|x, \mathbf{w}) = 1 - \sigma(\mathbf{w}^T x)$$

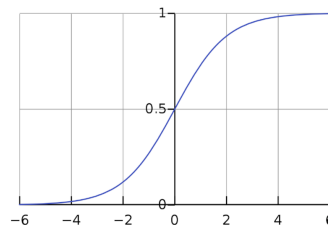
- We need to look for a function which gives us an output in the range $[0, 1]$. (like a probability).
- Let's denote this function with $\sigma(\cdot)$ and call it the **activation function**.

Introduction (cont.)

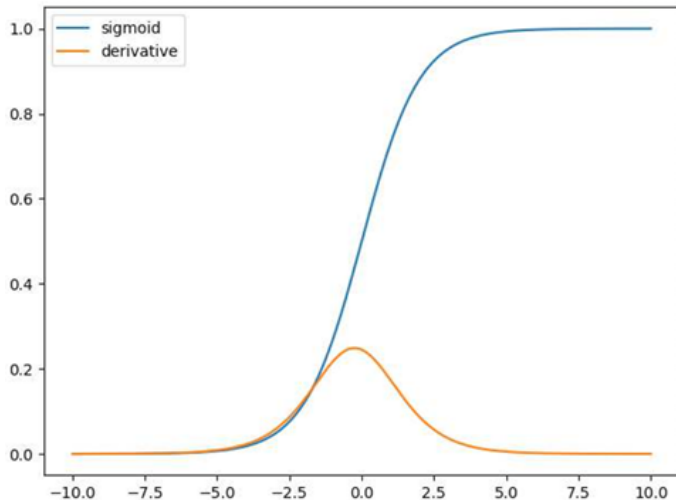
- Sigmoid (logistic) function.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- A good candidate for activation function.
- It gives us a number between 0 and 1 **smoothly**.
- It is also **differentiable**



Sigmoid function & its derivative



Introduction (cont.)

- The sigmoid function takes a number as input but we have:

$$x = [x_0 = 1, x_1, \dots, x_d]$$

$$w = [w_0, w_1, \dots, w_d]$$

- So we can use the **dot product** of x and w .
- We have $0 \leq \sigma(\mathbf{w}^T x) \leq 1$. which is the estimated probability of $y = 1$ on input x .
- An Example : A basketball game (Win, Lose)
 - $\sigma(\mathbf{w}^T x) = 0.7$
 - In other terms 70 percent chance of winning the game.

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

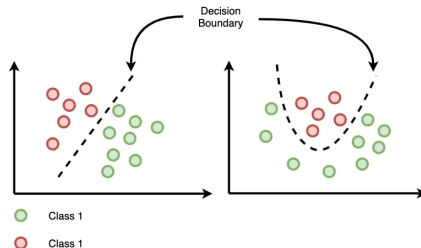
3 Summary

4 Extra reading

5 References

Decision surface

- Decision surface or decision boundary is the region of a problem space in which the output label of a classifier is ambiguous. (could be linear or non-linear)
- In binary classification it is where the probability of a sample belonging to each $y = 0$ and $y = 1$ is equal.



- Decision boundary hyperplane always has **one less dimension** than the feature space.

Decision surface (cont.)

- An example of linear decision boundaries:

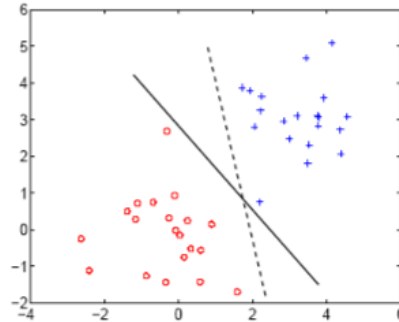
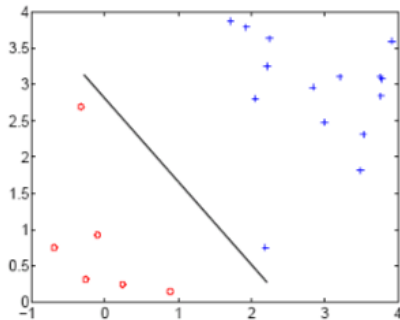


Figure adapted from Eric Xing, Machine Learning, CMU

Decision surface (cont.)

- Back to our logistic regression problem.
- Decision surface $\sigma(\mathbf{w}^T x) = \mathbf{constant}$.

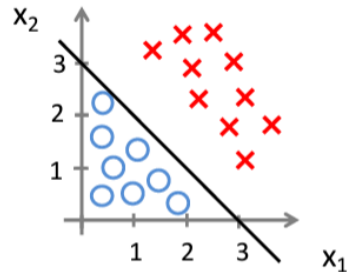
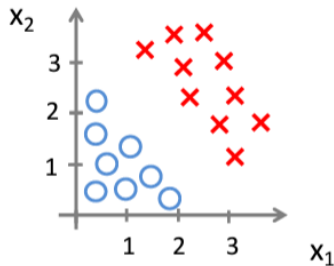
$$\sigma(\mathbf{w}^T x) = \frac{1}{1 + e^{-(\mathbf{w}^T x)}} = 0.5$$

- Decision surfaces are **linear functions** of x
 - if $\sigma(\mathbf{w}^T x) \geq 0.5$ then $\hat{y} = 1$, else $\hat{y} = 0$
 - Equivalently, if $\mathbf{w}^T x + w_0 \geq 0.5$ then decide $\hat{y} = 1$, else $\hat{y} = 0$

$\hat{y}$ is the predicted label

Decision boundary example

$$\sigma(\mathbf{w}^T x) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$$

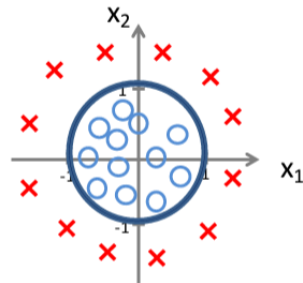
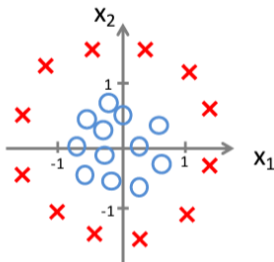


Predict $y = 1$ if $-3 + x_1 + x_2 \geq 0$

Non-linear decision boundary example

$$\sigma(\mathbf{w}^T x) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2)$$

We can learn more complex decision boundaries when having higher order terms



$$\text{Predict } y = 1 \text{ if } -1 + x_1^2 + x_2^2 \geq 0$$

① Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation



Cost function

Gradient descent

Multi-class logistic regression

③ Summary

④ Extra reading

⑤ References

ML estimation

- We had posterior of a sample as:

$$P(y^{(i)} | x^{(i)}, \mathbf{w})$$

- Logistic regression should maximize production of all these sample posteriors.
- Maximum (conditional) log likelihood:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmax}} \log \prod_{i=1}^n P(y^{(i)} | x^{(i)}, \mathbf{w})$$

- Note that in **binary** classification y is either 1 or 0, So we can have posterior term simplified as follows:

$$P(y^{(i)} | x^{(i)}, \mathbf{w}) = \sigma(\mathbf{w}^T x^{(i)})^{y^{(i)}} (1 - \sigma(\mathbf{w}^T x^{(i)}))^{(1-y^{(i)})}$$

ML estimation

- Logarithm of the posterior probability:

$$\log P(y^{(i)} | x^{(i)}, \mathbf{w}) = y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) + (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))$$

- Hence the log likelihood is as follows:

$$\begin{aligned} \log \prod_{i=1}^n P(y^{(i)} | x^{(i)}, \mathbf{w}) &= \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}, \mathbf{w}) \\ &= \sum_{i=1}^n [y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) + (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))] \end{aligned}$$

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

3 Summary

4 Extra reading

5 References

Cost function

- We should find

$$\hat{\mathbf{w}} = \underset{w}{\operatorname{argmin}} J(w)$$

- MLE finds parameters that best describe a classification problem so cost function should be negative of log likelihood term:


$$\begin{aligned} J(w) &= - \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}, \mathbf{w}) \\ &= \sum_{i=1}^n -y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)})) \end{aligned}$$

- No closed form solution for $\nabla_w J(w) = 0$
- However $J(w)$ is **convex**.

Cost function (cont.)

- Convexity of $J(w)$ can easily be proved:
 - We use the lemma that sum of several convex functions is still convex (you can prove it on your own).
 - Each term in the summation is differentiable (twice).
 - If you twice get derivative of (with respect to σ):

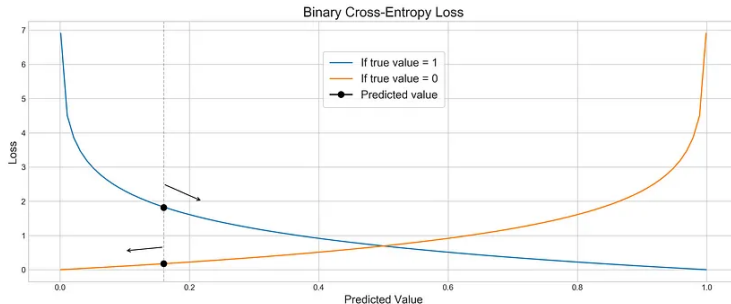
$$-y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))$$

- You get:

$$\frac{y}{\sigma^2} + \frac{1-y}{(1-\sigma)^2}$$

- Which for both $y = 0$ and $y = 1$ is positive.
- Each $\log P(y^{(i)} | x^{(i)}, \mathbf{w})$ is convex, hence the summation is convex as well.

- Visualization of each binary cross entropy loss term:



- As you can see if the model predicted value is $\hat{y} = 0.16$ and true label is $y = 1$ then the error is high but if the true label is $y = 0$ the error would be low.

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

3 Summary

4 Extra reading

5 References

Gradient descent

- Remember from previous slides:

$$J(w) = \sum_{i=1}^n -y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))$$

- Update rule for **gradient descent**:

$$w^{t+1} = w^t - \eta \nabla_w J(w^t)$$

- With $J(w)$ definition for logistic regression we get:

$$\nabla_w J(w) = \sum_{i=1}^n (\sigma(\mathbf{w}^T x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient descent

- Compare the gradient of **logistic regression** with the gradient of **SSE** in **linear regression** :

$$\nabla_w J(w) = \sum_{i=1}^n (\sigma(\mathbf{w}^T x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\nabla_w J(w) = \sum_{i=1}^n (\mathbf{w}^T x^{(i)} - y^{(i)}) x^{(i)}$$

Loss function

- Loss function is a single overall measure of loss incurred for taking our decisions (over entire dataset).
- We have:

$$Loss(y, \sigma(\mathbf{w}^T x)) = -y \times \log(\sigma(\mathbf{w}^T x)) - (1 - y) \times \log(1 - \sigma(\mathbf{w}^T x))$$

- Since in binary classification either $y = 1$ or $y = 0$ we have:

$$Loss(y, \sigma(\mathbf{w}^T x)) = \begin{cases} -\log(\sigma(\mathbf{w}^T x)) & \text{if } y = 1 \\ -\log(1 - \sigma(\mathbf{w}^T x)) & \text{if } y = 0 \end{cases}$$

- How is it related to zero-one loss? ($\hat{y}$ is the predicted label and y is the true label)

$$Loss(y, \hat{y}) = \begin{cases} 1 & \text{if } y \neq \hat{y} \\ 0 & \text{if } y = \hat{y} \end{cases}$$

1 Introduction

② Logistic Regression

Fundamentals

Decision surface

ML estimation

Cost function

Gradient descent

Multi-class logistic regression

③ Summary

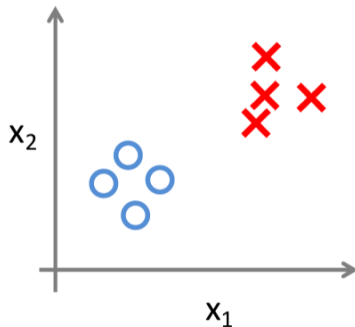
4 Extra reading

5 References

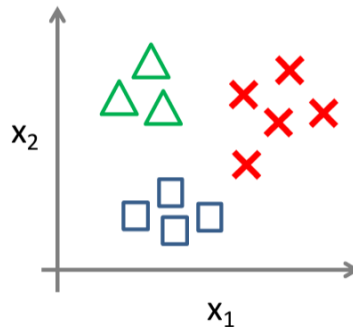
Multi-class logistic regression

- Now consider a problem where we have K classes and every sample only belongs to one class (for simplicity).

Binary classification:



Multi-class classification:



Multi-class logistic regression (cont.)

- For each class k , $\sigma_k(x; \mathbf{W})$ predicts the probability of $y = k$.
 - i.e., $P(y = k|x, \mathbf{W})$
- For each data point x_0 , $\sum_{k=1}^K P(y = k|x_0, \mathbf{W})$ must be 1
 - W denotes a matrix of w_i 's in which each w_i is a weight vector dedicated for class label i .
- On a new input x , to make a prediction, we pick the class that maximizes $\sigma_k(x; \mathbf{W})$:

$$\alpha(x) = \arg \max_{k=1, \dots, K} \sigma_k(x; \mathbf{W})$$

if $\sigma_k(x; \mathbf{W}) > \sigma_j(x; \mathbf{W}) \forall j \neq k$ then decide C_k

Multi-class logistic regression (cont.)

- $K > 2$ and $y \in \{1, 2, \dots, K\}$


$$\sigma_k(x, \mathbf{W}) = P(y = k|x) = \frac{\exp(w_k^T x)}{\sum_{j=1}^K \exp(w_j^T x)}$$

- Normalized exponential (Aka **Softmax**)
- if $w_k^T x \gg w_j^T x$ for all $j \neq k$ then $P(C_k|x) \approx 1$ and $P(C_j|x) \approx 0$
- Note : remember from Bayes theorem:

$$P(C_k|x) = \frac{P(x|C_k)P(C_k)}{\sum_{j=1}^K P(x|C_j)P(C_j)}$$

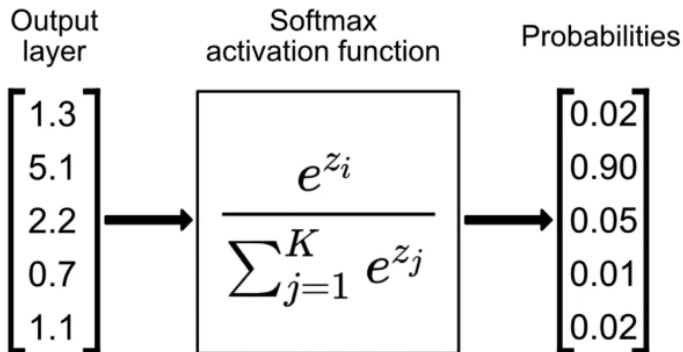
Multi-class logistic regression (cont.)

- Softmax function **smoothly** highlights the maximum probability and is differentiable.
- Compare it with `max(.)` function which is strict and non-differentiable
- Softmax can also handle negative values because we are using exponential function
- And it gives us probability for each class since:

$$\sum_{k=1}^K \frac{\exp(w_k^T x)}{\sum_{j=1}^K \exp(w_j^T x)} = 1$$

Multi-class logistic regression (cont.)

- An example of applying softmax (note that $z_i = w^T x_i$):



Multi-class logistic regression (cont.)

- Again we set $J(W)$ as negative of log likelihood.
- We need $\hat{W} = \underset{W}{\operatorname{argmin}} J(W)$

$$\begin{aligned} J(W) &= -\log \prod_{i=1}^n P(y^{(i)} | x^{(i)}, \mathbf{W}) \\ &= -\log \prod_{i=1}^n \prod_{k=1}^K \sigma_k(x^{(i)}; \mathbf{W})^{y_k^{(i)}} \\ &= -\sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log(\sigma_k(x^{(i)}; \mathbf{W})) \end{aligned}$$

- If **i-th** sample belongs to class k then $y_k^{(i)}$ is 1 else 0.
- Again no closed-form solution for $\hat{W}$

Multi-class logistic regression (cont.)

- From previous slides we have:

$$J(W) = - \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log(\sigma_k(x^{(i)}; \mathbf{W}))$$

- In which:

$$W = [w_1, w_2, \dots, w_K], \quad Y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & \dots & y_K^{(1)} \\ y_1^{(2)} & \dots & y_K^{(2)} \\ \vdots & \ddots & \vdots \\ y_1^{(n)} & \dots & y_K^{(n)} \end{pmatrix}$$

- y is a vector of length K (1-of- K encoding)
 - For example $y = [0, 0, 1, 0]^T$ when the target class is C_3 .

Multi-class logistic regression (cont.)

- Update rule for gradient descent:

$$w_j^{t+1} = w_j^t - \eta \nabla_w J(W^t)$$
$$\nabla_{w_j} J(W) = \sum_{i=1}^n (\sigma_j(x^{(i)}; \mathbf{W}) - y_j^{(i)}) x^{(i)}$$

- w_j^t denotes the weight vector for class j (since in multi-class LR, each class has its own weight vector) in the t -th iteration

- 1 Introduction
- 2 Logistic Regression
- 3 Summary**
- 4 Extra reading
- 5 References

Logistic regression (LR) summary

- LR is a **linear** classifier
- LR optimization problem is obtained by **maximum likelihood**
- No closed-form solution for its optimization problem
 - But convex cost function and global optimum can be found by gradient ascent

1 Introduction

② Logistic Regression

③ Summary

④ Extra reading

Probabilistic view in classification

Probabilistic classifiers

5 References

- ## 1 Introduction

- ## 2 Logistic Regression

- ### ③ Summary

- #### 4 Extra reading

Probabilistic view in classification

Probabilistic classifiers

- ## 5 References

Probabilistic view in classification problem

- In a classification problem:
 - Each **feature** is a **random variable** (e.g. a person's height)
 - The **class label** is also considered a **random variable** (e.g. a person could be overweight or not)
- We observe the feature values for a random sample and intend to find its class label
 - Evidence: Feature vector x
 - Objective: Class label

Definitions

- Posterior probability : The probability of a class label C_k given a sample x

$$P(C_k|x)$$

- Likelihood or class conditional probability : PDF of feature vector x for samples of class C_k

$$P(x|C_k)$$

- Prior probability : Probability of the label be C_k

$$P(C_k)$$

- $P(x)$: PDF of feature vector x
 - From total probability theorem:

$$P(x) = \sum_{k=1}^K P(x|C_k)P(C_k)$$

- 1 Introduction
- 2 Logistic Regression
- 3 Summary
- 4 Extra reading
 - Probabilistic view
 - Probabilistic classification
- 5 References

Probabilistic classifiers

- Probabilistic approaches can be divided in two main categories:
 - Generative
 - Estimate PDF $P(x, C_k)$ for each class C_k and then use it to find $P(C_k|x)$. Alternatively estimate both PDF $P(x|C_k)$ and $P(C_k)$ to find $P(C_k|x)$.
 - Discriminative
 - Directly estimate $P(C_k|x)$ for class C_k

Probabilistic classifiers (cont.)

- Let's assume we have input data x and want to classify the data into labels y .
- A generative model learns the **joint** probability distribution $P(x, y)$.
- A discriminative model learns the **conditional** probability distribution $P(y|x)$

- $v=0$ $v=1$

- $D(x|y)$ is:

- $$v \equiv 0 \quad v \equiv 1$$

Discriminative vs. Generative : example (cont.)

- The distribution $P(y|x)$ is the natural distribution for classifying a given sample x into class y .
 - This is why that algorithms which model this directly are called **discriminative** algorithms.
- Generative algorithms model $P(x, y)$, which can be transformed into $P(y|x)$ by Bayes rule and then used for classification.
 - However, the distribution $P(x, y)$ can also be used for other purposes.
 - For example we can use $P(x, y)$ to **generate** likely (x, y) pairs

Generative approach

1 Inference

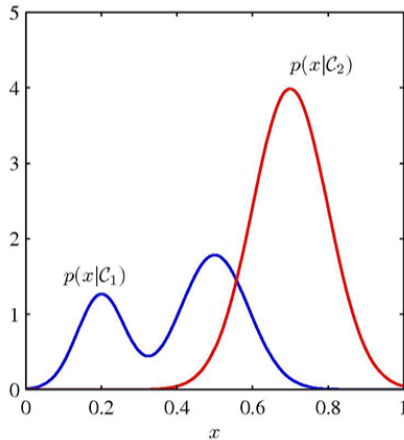
- Determine class conditional densities $P(x|C_k)$ and priors $P(C_k)$
- Use Bayes theorem to find $P(C_k|x)$

② Decision

- Make optimal assignment for new input (after learning the model in the inference stage)
- if $P(C_i|x) > P(C_j|x) \forall j \neq i$, then decide C_i .

Generative approach (cont.)

- Generative approach for a binary classification problem:



Figures adapted from Machine Learning and Pattern Recognition, Bishop

Discriminative approach

1 Inference

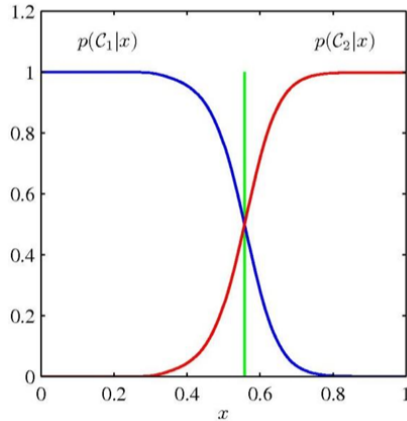
- Determine the posterior class probabilities $P(C_k|x)$ directly.

② Decision

- Make optimal assignment for new input (after learning the model in the inference stage)
- if $P(C_i|x) > P(C_j|x) \forall j \neq i$, then decide C_i .

Discriminative approach (cont.)

- Discriminative approach for a binary classification problem:



Figures adapted from Machine Learning and Pattern Recognition, Bishop

Discriminative approach (cont.)

- Logistic regression is a **discriminative** approach.
- We directly want to specify the class label with $\sigma(\mathbf{w}^T x)$

- 1 Introduction
- 2 Logistic Regression
- 3 Summary
- 4 Extra reading
- 5 References**

Contributions

- **These slides are authored by:**
 - Danial Gharib

- [1] M. Soleymani Baghshah, “Machine learning.” Lecture slides.
- [2] A. Ng, “ML-005, lecture 6.” Lecture slides.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Information Science and Statistics, New York, NY: Springer, 1 ed., Aug. 2006.
- [4] S. Fidler, “Csc411.” Lecture slides.
- [5] A. Ng and T. Ma, *CS229 Lecture Notes*. Updated June 11, 2023.

Any Questions?